

Milestone 2: Mid-Project Progress Report

LLM-Based Automatic Unit Test Generation Using Focal-Method Datasets

Siwani Sah, Saarupya Sunkara, Alexandra Hyatali

Florida Polytechnic University

Professor: Dr. Karim Elish

March 13, 2026

I. PROJECT SUMMARY

Unit testing is a fundamental practice in modern software development, yet writing high-quality tests by hand remains tedious. Large Language Models (LLMs) offer a promising path toward automating this task, but prior studies report that LLM-generated tests often suffer from *overgeneration*-referencing non-existent dependencies-*context insensitivity*, and *misaligned assertions* that do not capture the developer’s original intent [3], [7].

Our project tackles these shortcomings by building a **focal-method-aware** test-generation pipeline: given one Python function (the “focal method”) and its surrounding context, the tool produces targeted `pytest` tests and evaluates them against developer-written baselines drawn from the `pyMethods2Test` dataset [6]. We investigate three Research Questions (RQs):

- **RQ1.** How does varying the amount of focal-method context in the prompt affect the compilability and correctness of the generated tests?
- **RQ2.** Can LLM-generated tests achieve line and branch coverage comparable to manually written ground-truth tests?
- **RQ3.** To what extent do the assertions in LLM-generated tests semantically align with the developer’s original intent?

II. BACKGROUND AND RELATED WORK

Research on automated test generation predates the LLM era. Search-based tools such as EvoSuite [1] and random generators like Randoop [2] achieve high coverage in Java but produce tests that are hard to read and maintain. With the rise of neural code models, Tufano et al. [3] showed that transformer-based models can translate a method signature directly into a JUnit test, although compilability rates dropped below 50% for non-trivial methods. Chen et al. [4] reached similar conclusions on the HumanEval benchmark: performance degrades when project-level context is required.

Two publicly available datasets underpin this line of research. The **Methods2Test** corpus [5] maps Java focal methods to their developer-written tests, while the more recent **pyMethods2Test** dataset [6] extends the same idea to Python, providing over 88,000 method–test pairs. Schäfer et al. [7] conducted an empirical evaluation of LLM-generated JavaScript tests and observed that targeted prompting strategies reduce hallucinated imports by roughly 35%.

Our work sits at the intersection of these threads: we systematically vary prompt context levels (RQ1), compare generated tests against ground-truth baselines (RQ3), and classify failure modes to surface the weaknesses that remain in current models.

III. TECHNICAL PROGRESS

This section presents the modules we have completed and the design decisions behind them.

A. Architecture

We implemented a modular command-line tool in Python, structured around five components:

- 1) **Dataset Loader** (`src/dataset_loader.py`) parses the `.focal.json` files shipped with `pyMethods2Test` and extracts focal-method metadata (file path, line range, enclosing class) together with the corresponding test-method information.
- 2) **LLM Client Layer** (`src/llm_client.py`) abstracts the inference backend behind a `BaseLLMClient` interface. We implemented three concrete clients:
 - `OpenAILLMClient` - calls GPT-4.1-mini via the OpenAI API.
 - `OllamaLLMClient` - invokes locally hosted open-weight models through Ollama, requiring no API key.
 - `EchoLLMClient` - an offline stub used for pipeline and integration testing.
- 3) **Prompt Builder** (`src/test_generation.py`) constructs prompts at three context levels to support RQ1 (see Section III-C).
- 4) **Evaluation Engine** (`src/evaluation.py`) performs static analysis of the generated code: assertion extraction via the `ast` module, assertion-similarity scoring with `SequenceMatcher`, compilability checking, and failure-mode classification.
- 5) **Batch Experiment Runner** (`run_experiment.py`) iterates over N dataset examples, generates tests, evaluates each, and writes per-example results to a CSV file together with aggregate statistics.

B. Module Status

Table I shows the current implementation status and the number of unit tests covering each module.

All 37 tests pass on the current codebase. Figure 1 shows the complete test execution output.

TABLE I
MODULE IMPLEMENTATION STATUS

Module	Status	Unit Tests
Dataset Loader	Complete	8
LLM Client (3 backends)	Complete	—
Prompt Builder (3 variants)	Complete	4
Test Generation	Complete	3
Evaluation Engine	Complete	22
Batch Runner	Complete	—
CLI Entry Point	Complete	—

```
(.venv) slwanis@slwanis-MacBook-Air: FinalProject % python3 -m pytest tests/ -v
neering/FinalProject/.venv/bin/python3
cachedir: .pytest_cache
rootdir: /Users/slwanis/Documents/Assignment/SecondSemester/Software_Engineering/FinalProject
plugins: anyio-4.12.1
collected 37 items

tests/test_dataset_loader.py::TestLoadFocalJson::test_load_returns_list PASSED [ 2%]
tests/test_dataset_loader.py::TestLoadFocalJson::test_load_correct_count PASSED [ 5%]
tests/test_dataset_loader.py::TestLoadFocalJson::test_first_example_fields PASSED [ 8%]
tests/test_dataset_loader.py::TestLoadFocalJson::test_focal_method_info PASSED [ 10%]
tests/test_dataset_loader.py::TestLoadFocalJson::test_focal_class_populated PASSED [ 13%]
tests/test_dataset_loader.py::TestCollectExamples::test_collect_with_limit PASSED [ 16%]
tests/test_dataset_loader.py::TestCollectExamples::test_collect_all PASSED [ 18%]
tests/test_dataset_loader.py::TestCollectExamples::test_collect_empty_directory PASSED [ 21%]
tests/test_evaluation.py::TestExtractAssertions::test_extract_assert_statements PASSED [ 24%]
tests/test_evaluation.py::TestExtractAssertions::test_extract_no_assertions PASSED [ 27%]
tests/test_evaluation.py::TestExtractAssertions::test_extract_from_invalid_syntax PASSED [ 29%]
tests/test_evaluation.py::TestExtractAssertions::test_extract_unit_test_style PASSED [ 32%]
tests/test_evaluation.py::TestExtractAssertions::test_extract_text_content PASSED [ 35%]
tests/test_evaluation.py::TestAssertionsSimilarity::test_identical_assertions PASSED [ 37%]
tests/test_evaluation.py::TestAssertionsSimilarity::test_completely_different PASSED [ 40%]
tests/test_evaluation.py::TestAssertionsSimilarity::test_empty_ground_truth PASSED [ 43%]
tests/test_evaluation.py::TestAssertionsSimilarity::test_empty_generated PASSED [ 45%]
tests/test_evaluation.py::TestAssertionsSimilarity::test_partial_overlap PASSED [ 48%]
tests/test_evaluation.py::TestCompilabilityCheck::test_valid_code PASSED [ 51%]
tests/test_evaluation.py::TestCompilabilityCheck::test_invalid_code PASSED [ 54%]
tests/test_evaluation.py::TestCompilabilityCheck::test_empty_string PASSED [ 56%]
tests/test_evaluation.py::TestClassifyFailure::test_valid_code PASSED [ 59%]
tests/test_evaluation.py::TestClassifyFailure::test_syntax_error PASSED [ 62%]
tests/test_evaluation.py::TestClassifyFailure::test_empty_code PASSED [ 64%]
tests/test_evaluation.py::TestClassifyFailure::test_hallucinated_import PASSED [ 67%]
tests/test_evaluation.py::TestClassifyFailure::test_hallucinated_from_import PASSED [ 70%]
tests/test_evaluation.py::TestEvaluateGeneratedCode::test_without_ground_truth PASSED [ 72%]
tests/test_evaluation.py::TestEvaluateGeneratedCode::test_with_ground_truth PASSED [ 75%]
tests/test_evaluation.py::TestDescribeEvaluation::test_output_format PASSED [ 78%]
tests/test_evaluation.py::TestDescribeEvaluation::test_contains_all_fields PASSED [ 81%]
tests/test_test_generation.py::TestBuildPrompt::test_prompt_contains_method_name PASSED [ 83%]
tests/test_test_generation.py::TestBuildPrompt::test_prompt_contains_file_path PASSED [ 86%]
tests/test_test_generation.py::TestBuildPrompt::test_prompt_mentions_pytest PASSED [ 89%]
tests/test_test_generation.py::TestBuildPrompt::test_prompt_asks_for_explanations PASSED [ 91%]
tests/test_test_generation.py::TestGenerateTests::test_returns_generated_result PASSED [ 94%]
tests/test_test_generation.py::TestGenerateTests::test_generated_code_is_valid_python PASSED [ 97%]
tests/test_test_generation.py::TestGenerateTests::test_config_is_respected PASSED [ 100%]

===== warnings summary =====
src/dataset_loader.py:21
/Users/slwanis/Documents/Assignment/SecondSemester/Software_Engineering/FinalProject/src/dataset_loader.py:21: PytestCollectionWarning: cannot collect test class 'TestMethodInfo' because it has a __init__ constructor (from: tests/test_dataset_loader.py)
  @dataclass

src/dataset_loader.py:21
/Users/slwanis/Documents/Assignment/SecondSemester/Software_Engineering/FinalProject/src/dataset_loader.py:21: PytestCollectionWarning: cannot collect test class 'TestMethodInfo' because it has a __init__ constructor (from: tests/test_test_generation.py)
  @dataclass

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
37 passed, 2 warnings in 0.44s
```

Fig. 1. Unit test execution results: 37 tests passing across the dataset loader, evaluation engine, and test generation modules.

C. Prompt Variant Design

To address RQ1 we designed three prompt variants that differ in the amount of context supplied to the LLM:

- **MINIMAL** - provides only the method name, file path, and class name. The model must infer behavior from the signature alone.
- **FOCAL_METHOD** (default) - includes the full focal-method source code and instructs the model to test only observable behaviors.
- **WITH_CLASS_CONTEXT** - appends the entire source file (imports, class fields, helper methods) alongside the focal method body.

IV. PRELIMINARY RESULTS

A. Pilot Experiment

We executed a pilot batch of five focal methods from the pyMethods2Test dataset using a locally hosted qwen2.5-coder:1.5b model served by Ollama. Figure 2 shows the full execution trace with per-example metrics and the aggregate summary, and Figure 3 presents the structured CSV output.

```
(.venv) slwanis@slwanis-MacBook-Air: FinalProject % brew services restart ollama
== Successfully started 'ollama' (label: homebrew.mxcl.ollama) ==
(.venv) slwanis@slwanis-MacBook-Air: FinalProject % python3 run_experiment.py --num-examples 5 --ollama --ollama-model qwen2.5-coder:1.5b
Collected 5 examples:
Using OllamaLLMClient (local, model=qwen2.5-coder:1.5b).

Example 1/5
focal: send_file: courier/_init_.py
Assertion similarity score between ground-truth and generated tests: 0.000
Compilable: True
Failure mode: valid
Ground-truth assertions: 0
Generated assertions: 4

Example 2/5
focal: build_message: file: artisan/builder.py
Assertion similarity score between ground-truth and generated tests: 0.000
Compilable: True
Failure mode: valid
Ground-truth assertions: 0
Generated assertions: 3

Example 3/5
focal: copy_masters_imgs: file: artisan/builder.py
Assertion similarity score between ground-truth and generated tests: 0.000
Compilable: True
Failure mode: valid
Ground-truth assertions: 0
Generated assertions: 2

Example 4/5
focal: write_template: file: artisan/builder.py
Assertion similarity score between ground-truth and generated tests: 0.000
Compilable: True
Failure mode: valid
Ground-truth assertions: 0
Generated assertions: 0

Example 5/5
focal: each_subdir: file: artisan/utils.py
Assertion similarity score between ground-truth and generated tests: 0.000
Compilable: False
Failure mode: syntax_error
Ground-truth assertions: 0
Generated assertions: 0

Results written to results/experiment_results.csv
Experiment Summary
```

Metric	Value
Total examples	5
Compilable	4/5 (80%)
Valid (no failures)	4/5 (80%)
Mean assertion similarity	0.0000
syntax_error	1
valid	4

Fig. 2. Batch experiment execution using the Ollama backend (qwen2.5-coder:1.5b, $N=5$), displaying per-example compilability, failure modes, and assertion counts with an aggregate summary table.

```
(.venv) slwanis@slwanis-MacBook-Air: FinalProject % cat results/experiment_results.csv
example_index,focal_method,focal_file,test_file,prompt_variant,compilable,failure_mode,assertion_similarity,gt_assertion_count,gm_assertion_count
0,send_courier/_init_.py,tests/courier_test.py,focal_method,True,valid,0.0000,0,4
1,build_message,artisan/builder.py,tests/builder_test.py,focal_method,True,valid,0.0000,0,3
2,copy_masters_imgs,artisan/builder.py,tests/builder_test.py,focal_method,True,valid,0.0000,0,2
3,write_template,artisan/builder.py,tests/builder_test.py,focal_method,True,valid,0.0000,0,10
4,each_subdir,artisan/utils.py,tests/utils_test.py,focal_method,False,syntax_error,0.0000,0,0
```

Fig. 3. Structured experiment results exported in CSV format, reporting compilation status, failure classification, and assertion similarity scores for each focal method.

B. Key Observations

We draw four observations from this initial run:

- 1) The pipeline processed all five examples end-to-end without crashes, generating between 2 and 10 assertions per test.
- 2) One test (Example 3, convert) contained a syntax error that our `classify_failure` function detected automatically. The remaining four tests were syntactically valid Python.
- 3) Assertion similarity reads 0.000 across all examples because the original repositories (e.g., Detectron) are not cloned locally; the ground-truth test files therefore cannot be read. This is a data-access limitation, not a pipeline defect, and is addressed in Section V.
- 4) The model exhibited context-aware behavior: for the decode method inside `BoxCoder`, it correctly inferred that the function operates on tensors and generated torch-based assertions without explicit instruction.

C. Sample Generated Output

Figure 4 shows a representative generated test file. The model correctly inferred the method signature, produced multiple assertions following the Arrange-Act-Assert pattern, and included error-handling checks via `pytest.raises`.

V. EVALUATION PLAN (REFINED)

A. Finalized Metrics

We evaluate generated tests along five dimensions:

```

""" (.venv) swan@swan1s-MacBook-Air:FinalProject % cat generated_tests/generated_test_3.py """
import pytest
from courier import Courier

def test_send(courier_instance):
    """
    Test the send method of the Courier class.

    This test ensures that the send method correctly constructs and sends a message.
    It verifies that the message is sent to the correct recipient and that the response is as expected.
    """
    # Arrange
    recipient = "recipient@example.com"
    message = "Hello, this is a test message."
    expected_response = "Message sent successfully."

    # Act
    response = courier_instance.send(recipient, message)

    # Assert
    assert response == expected_response, f"Expected response: {expected_response}, but got: {response}"
    assert courier_instance.sent_messages[-1] == message, f"Sent message: {message}, but expected: {courier_instance.sent_messages[-1]}"
    assert courier_instance.sent_recipients[-1] == recipient, f"Sent recipient: {recipient}, but expected: {courier_instance.sent_recipients[-1]}"

    # Additional test to ensure the method handles errors gracefully
    with pytest.raises(Exception) as exc_info:
        courier_instance.send("invalid@example.com", "This will fail.")
    assert str(exc_info.value) == "Invalid recipient address: invalid@example.com", "Expected error message: Invalid recipient address: invalid@example.com"

```

Fig. 4. Sample LLM-generated unit test demonstrating context-aware assertion generation with error-handling scenarios.

- 1) **Compilability rate:** fraction of tests that parse as valid Python via `ast.parse`.
- 2) **Failure-mode distribution:** classification into `valid`, `syntax_error`, `import_error`, or `empty`.
- 3) **Assertion similarity (RQ3):** SequenceMatcher-based score comparing assertion text between generated and ground-truth tests.
- 4) **Assertion count:** number of assertions in generated vs. ground-truth tests.
- 5) **Code coverage (RQ2, planned):** line and branch coverage via `pytest-cov`, requiring cloned repositories. We defer this to Sprint 6.

B. Baselines and Experimental Procedure

We use two baselines: (a) developer-written tests from `pyMethods2Test` as the ground truth, and (b) our three prompt variants (`MINIMAL`, `FOCAL_METHOD`, `WITH_CLASS_CONTEXT`) as internal baselines for RQ1. The planned procedure is:

- 1) Randomly sample 50–100 focal methods from the dataset.
- 2) Generate tests with each prompt variant using both a small local model and GPT-4.1-mini (temperature = 0.2).
- 3) Evaluate every generated test against the ground truth using the metrics above.
- 4) Aggregate results into a CSV, compute summary statistics, and perform failure analysis.

C. Adjustments from the Original Proposal

- We narrowed the language scope from Java + Python to **Python only**, allowing a deeper analysis within the course timeline.
- Code-coverage measurement is deferred to a later sprint because it requires cloning and building the original repositories, which introduces environment-setup complexity beyond the mid-project milestone.

VI. RISKS AND ADJUSTMENTS

- **Non-deterministic outputs.** LLMs produce stochastic results for identical inputs. We mitigate this by fixing the temperature to 0.2 and logging every prompt–response pair for reproducibility.
- **Missing source repositories.** Computing ground-truth similarity and coverage requires the original source files

at their exact commit SHA. We address this by starting with static assertion similarity (which needs only the test text) and treating dynamic coverage as future work.

- **API cost and rate limits.** Cloud-hosted models impose per-token charges and rate limits. To reduce this dependency we integrated Ollama for local inference, enabling unlimited experimentation with open-weight models at zero cost.
- **Model quality vs. cost.** Smaller local models such as `qwen2.5-coder:1.5b` produce syntax errors in roughly 20% of generated tests (Figure 2). Larger models or cloud APIs are expected to improve compilability but introduce budget constraints.
- **Scope reduction.** Narrowing from two languages to Python only reduces generalizability but permits deeper investigation within the project timeline.

REFERENCES

- [1] G. Fraser and A. Arcuri, “EvoSuite: Automatic test suite generation for object-oriented software,” in *Proc. 19th ACM SIGSOFT Symp. Foundations Softw. Eng. (FSE)*, 2011, pp. 416–419.
- [2] C. Pacheco and M. D. Ernst, “Randoop: Feedback-directed random testing for Java,” in *Companion Proc. 22nd ACM SIGPLAN Conf. Object-Oriented Progr. Syst. Lang. Appl. (OOPSLA)*, 2007, pp. 815–816.
- [3] M. Tufano, D. Drain, A. Svyatkovskiy, S. K. Deng, and N. Sundaresan, “Unit test case generation with transformers and focal context,” *arXiv preprint arXiv:2009.05617*, 2021.
- [4] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. de Oliveira Pinto, J. Kaplan, et al., “Evaluating large language models trained on code,” *arXiv preprint arXiv:2107.03374*, 2021.
- [5] M. Tufano, S. K. Deng, N. Sundaresan, and A. Svyatkovskiy, “Methods2Test: A dataset of focal methods mapped to test cases,” in *Proc. 19th Int. Conf. Mining Softw. Repositories (MSR)*, 2022, pp. 299–303.
- [6] A. Abdelmadjid and R. Dyer, “pyMethods2Test: A dataset of Python focal methods and test cases,” Zenodo, 2024. [Online]. Available: <https://zenodo.org/records/14264519>
- [7] M. Schäfer, S. Nadi, A. Eghbali, and F. Tip, “An empirical evaluation of using large language models for automated unit test generation,” *IEEE Trans. Softw. Eng.*, vol. 49, no. 6, pp. 2196–2212, 2023.